

Clase 6.2

Un *framework* para GNNs

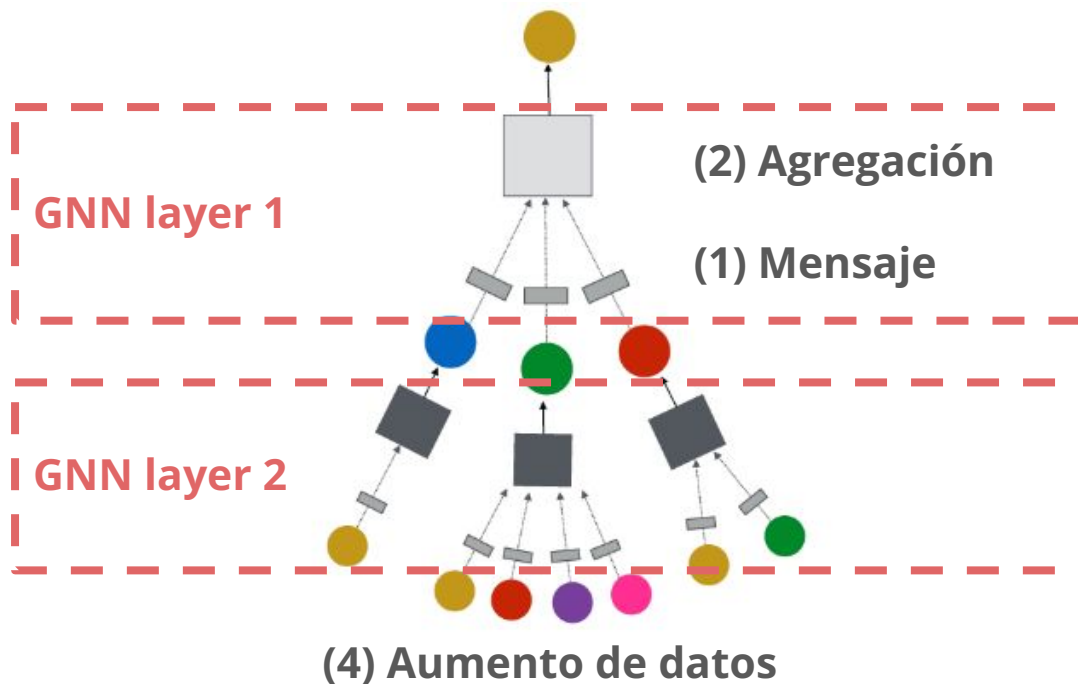
Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl

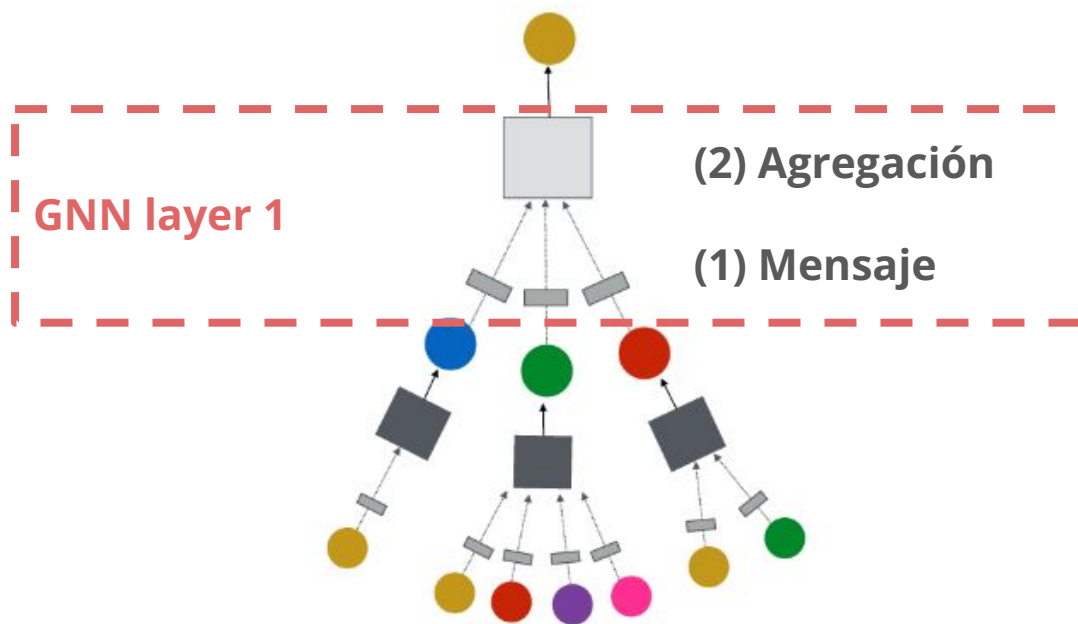
Un framework general para GNNs

(5) Objetivo de aprendizaje

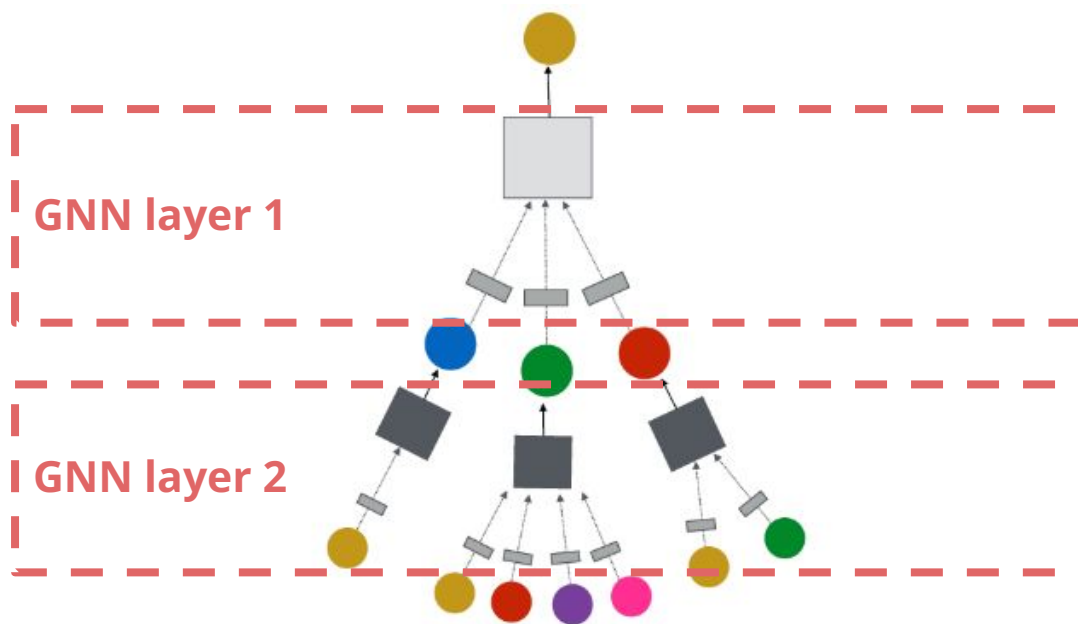
(3) Conexión
de capas



Capa GNN = Mensaje + Agregación

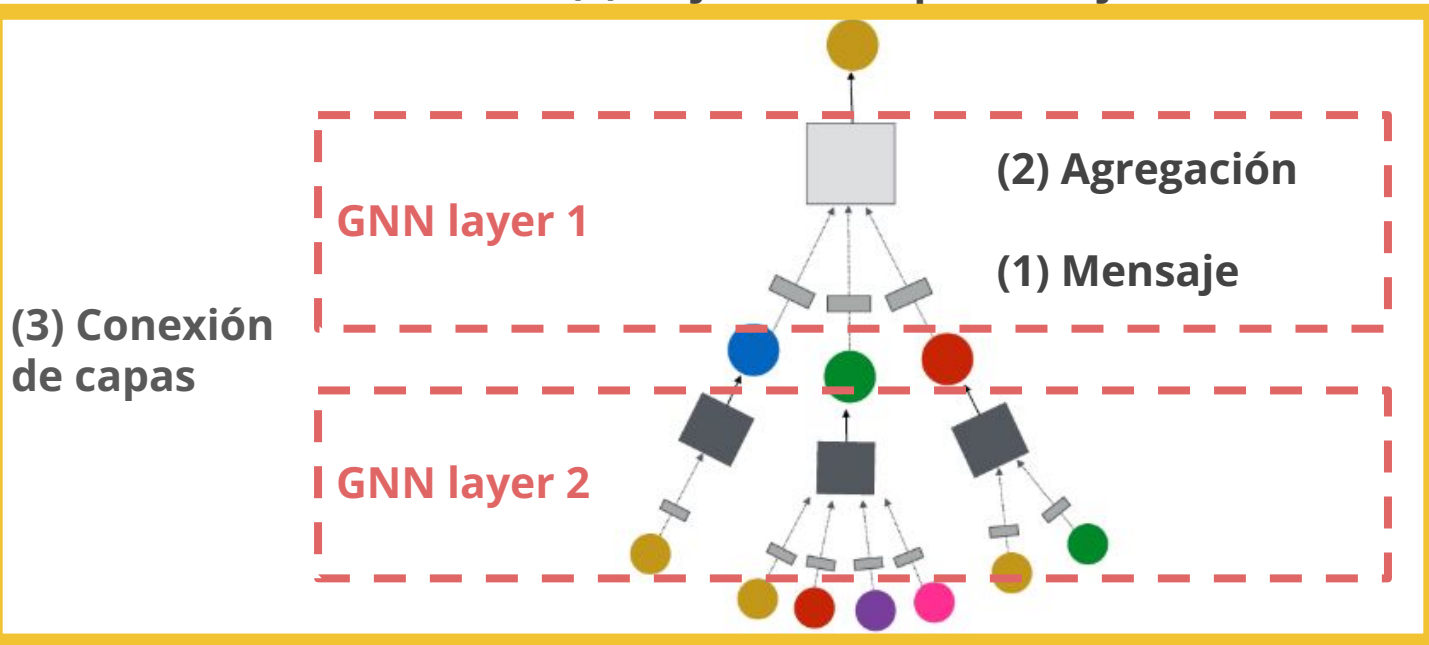


Conectando múltiples capas GNN



Un framework general para GNNs

(5) Objetivo de aprendizaje



Clase de hoy

Un framework general para GNNs

(5) Objetivo de aprendizaje

(2) Agregación

(1) Mensaje

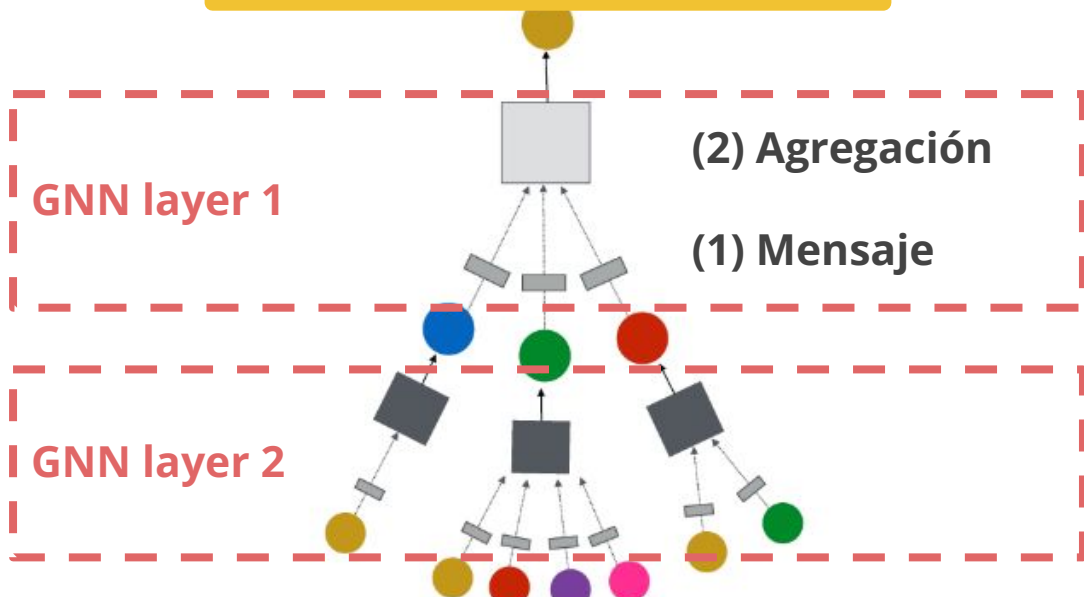
GNN layer 1

(3) Conexión de capas

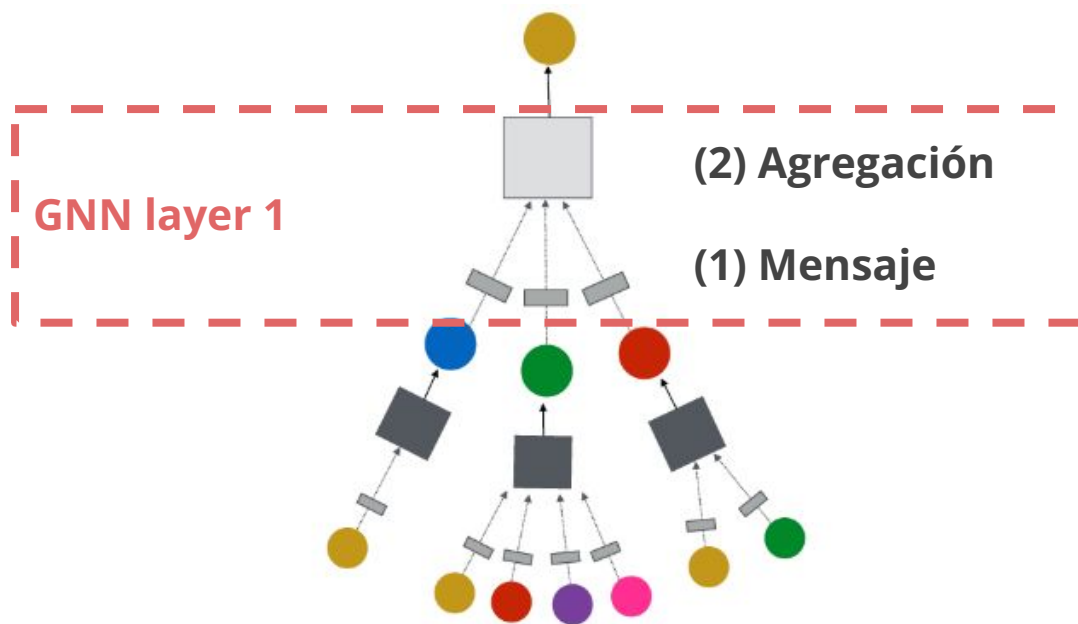
GNN layer 2

(4) Aumento de datos

Lo veremos de forma aplicada

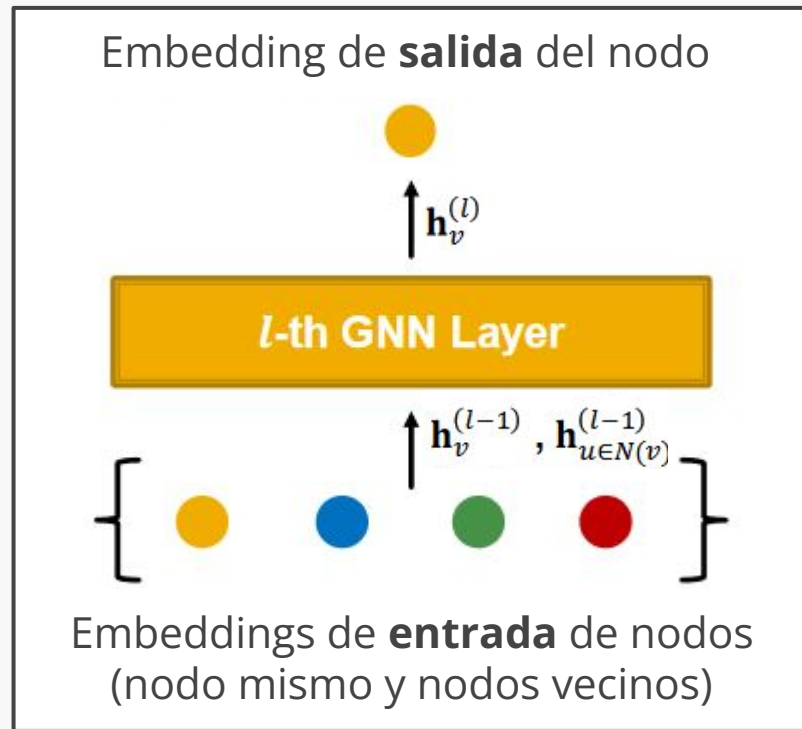


Capa GNN = Mensaje + Agregación



Idea de una capa GNN

- Comprimir múltiples vectores en uno solo
- Los dos pasos clave:
 1. Determinar el mensaje
 2. Agregación de mensajes



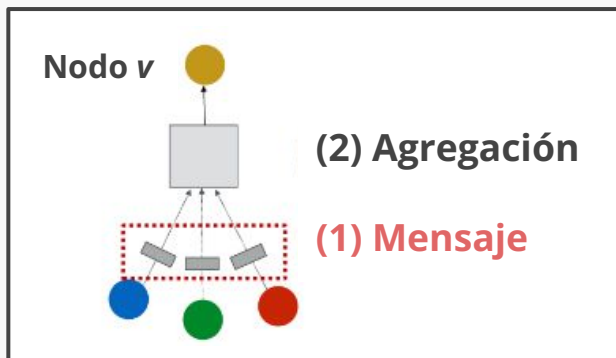
(1) Cálculo del mensaje

Idea: Cada nodo crea un mensaje que será enviado a otros nodos después

Ejemplo: Vimos que la GCN aplicaba una matriz de pesos a cada mensaje

$$\mathbf{m}_u^{(l)} = \text{MSG}^{(l)} \left(\mathbf{h}_u^{(l-1)} \right)$$

$$\mathbf{m}_u^{(l)} = \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}$$



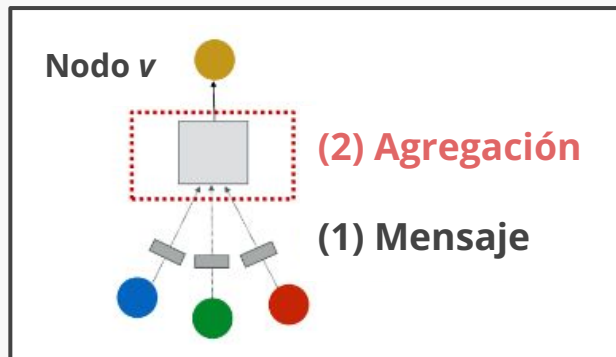
(2) Agregación de mensajes

Idea: Cada nodo agrega los mensajes de sus vecinos

Ejemplo: Algunas funciones agregadoras son suma, media o max.

$$\mathbf{h}_v^{(l)} = \text{AGG}^{(l)} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\} \right)$$

$$\mathbf{h}_v^{(l)} = \text{Sum}(\{\mathbf{m}_u^{(l)}, u \in N(v)\})$$



Problemas en la agregación de mensajes

Problema: La información de un nodo podría perderse, ya que el cálculo de $h_v^{(l)}$ no depende directamente de $h_v^{(l-1)}$.

Solución: Incluir el estado anterior en el cálculo

(1) **Mensaje:** calcular el mensaje del nodo mismo

(2) **Agregación:** después de agregar los vecinos, podemos agregar $m_v^{(l)}$

$$\mathbf{h}_v^{(l)} = \text{CONCAT} \left(\text{AGG} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\} \right), \mathbf{m}_v^{(l)} \right)$$

(3) Actualización de representación

Problema: La información de un nodo podría perderse, ya que el cálculo de $h_v^{(l)}$ no depende directamente de $h_v^{(l-1)}$.

Solución: Incluir el estado anterior en el cálculo

(1) **Mensaje:** calcular el mensaje del nodo mismo

(2) **Agregación:** después de agregar los vecinos, podemos agregar $m_v^{(l)}$

$$\mathbf{h}_v^{(l)} = \text{CONCAT} \left(\text{AGG} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\} \right), \mathbf{m}_v^{(l)} \right)$$

Una sola capa GNN

Con una sola capa, nos enfocamos en el paso de mensajes:

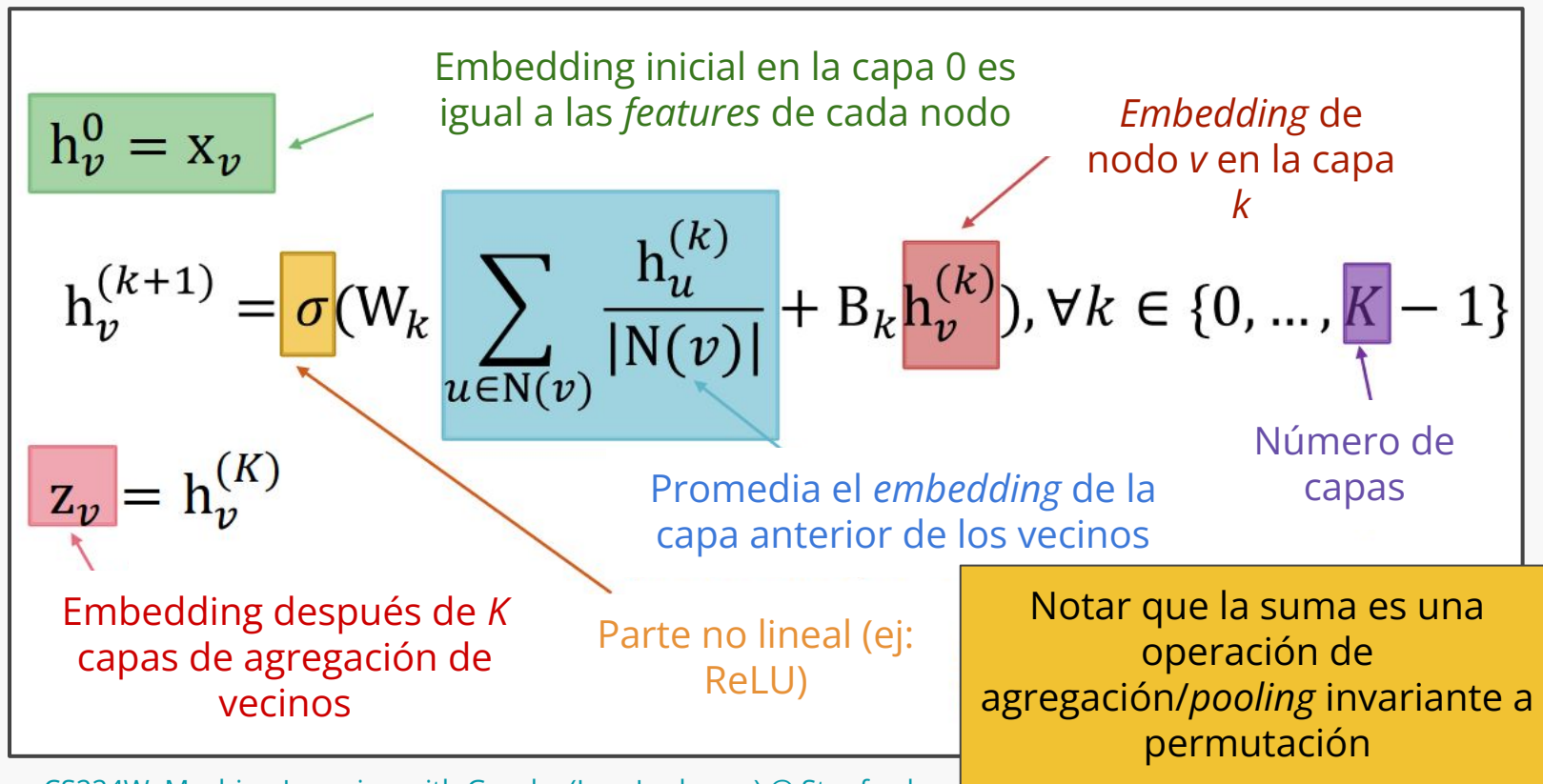
1. **Mensaje:** cada nodo calcula un mensaje

$$\mathbf{m}_u^{(l)} = \text{MSG}^{(l)} \left(\mathbf{h}_u^{(l-1)} \right), u \in \{N(v) \cup v\}$$

2. **Agregación:** agrega mensajes de los vecinos

$$\mathbf{h}_v^{(l)} = \text{AGG}^{(l)} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\}, \mathbf{m}_v^{(l)} \right)$$

Formulación matemática



Tarea a nivel de subgrafos

Ejemplo:
Predicción de tráfico vehicular

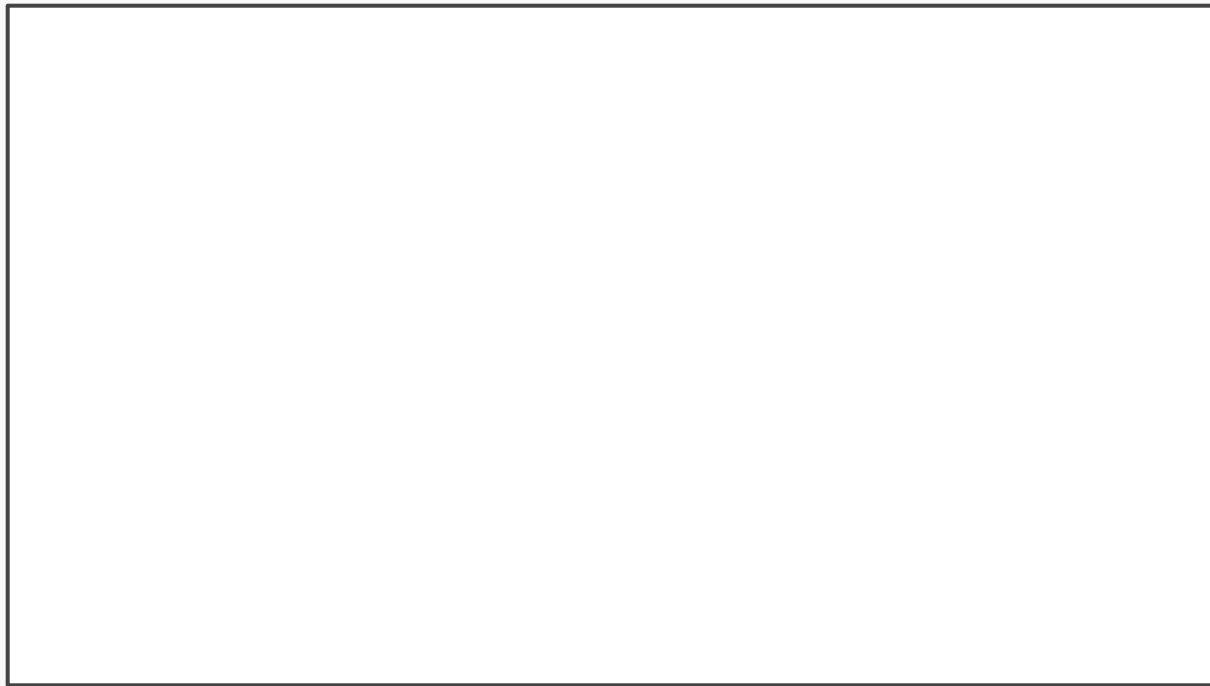
Uso de grafos para
modelamiento caminos y sus
posibles estados

Predicción de tiempos de llegada en Google Maps

Nodo: Segmentos de calle/camino

Vértices: Conectividad entre segmentos de calle/camino

Predicción: Tiempo estimado de llegada (ETA)



Identifiquemos cómo funciona

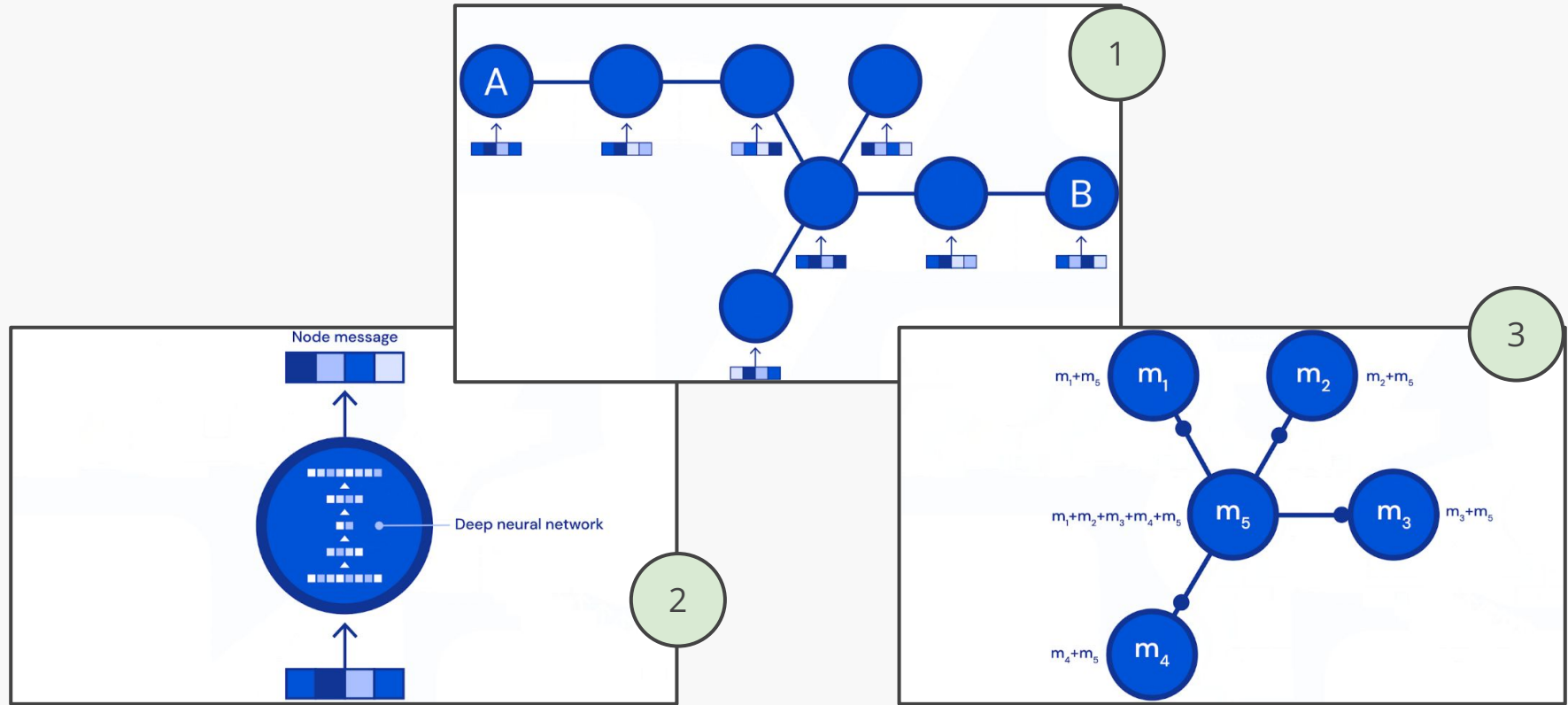
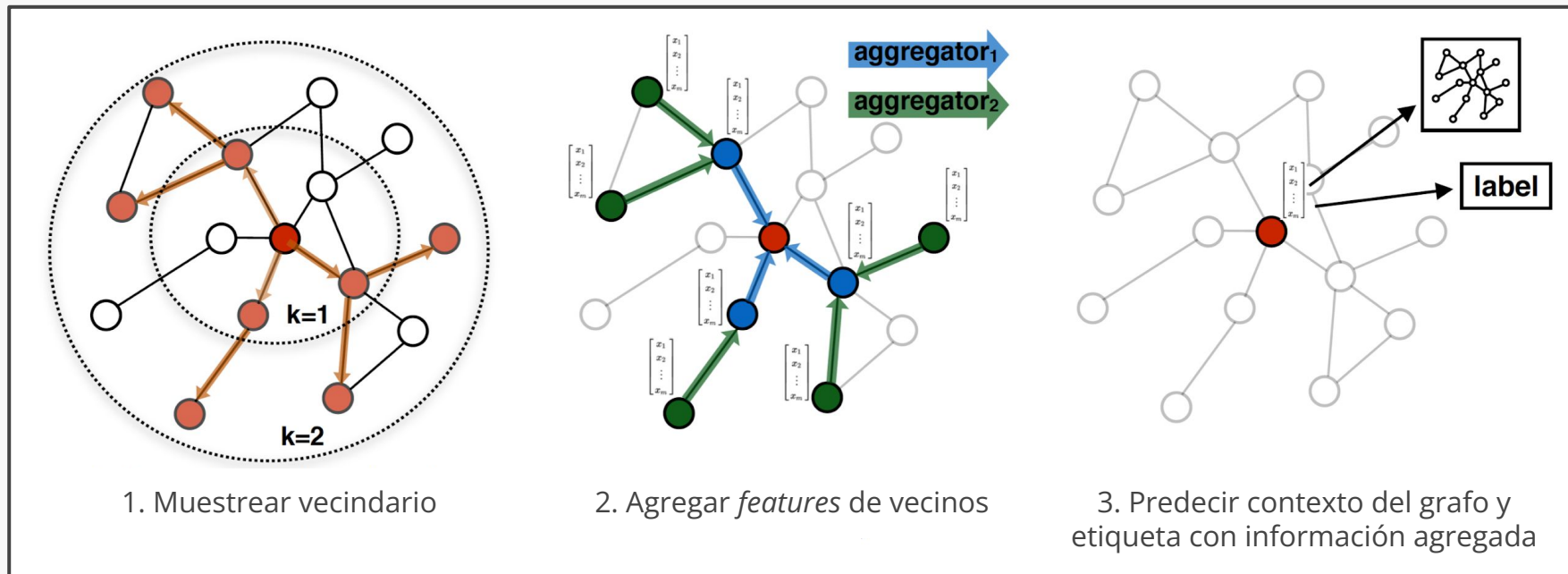


Figura: [Traffic prediction with advanced Graph Neural Networks - Google DeepMind](#)

Código*#GraphConv*

GNN clásicas: GraphSAGE



GNN clásicas: GraphSAGE

$$\mathbf{h}_v^{(l)} = \sigma \left(\mathbf{W}^{(l)} \cdot \text{CONCAT} \left(\mathbf{h}_v^{(l-1)}, \text{AGG} \left(\left\{ \mathbf{h}_u^{(l-1)}, \forall u \in N(v) \right\} \right) \right) \right)$$

- **Mensaje** se calcula dentro de AGG
- **Agregación** en dos etapas:
 - **Etapas 1:** Agregar los vecinos

$$\mathbf{h}_{N(v)}^{(l)} \leftarrow \text{AGG} \left(\left\{ \mathbf{h}_u^{(l-1)}, \forall u \in N(v) \right\} \right)$$

- **Etapas 2:** Agregar etapa 1 con la representación del mismo nodo

$$\mathbf{h}_v^{(l)} \leftarrow \sigma \left(\mathbf{W}^{(l)} \cdot \text{CONCAT}(\mathbf{h}_v^{(l-1)}, \mathbf{h}_{N(v)}^{(l)}) \right)$$

GNN clásicas: GraphSAGE

La función de agregación puede calcular el promedio (**mean**), aplicar algún *pooling* (**mean**, **max**), o incluso una LSTM

$$\text{AGG} = \sum_{u \in N(v)} \mathbf{h}_u^{(l-1)} \quad \text{Cálculo de mensaje}$$

Agregación

$$\text{AGG} = \text{Mean}(\{\text{MLP}(\mathbf{h}_u^{(l-1)}), \forall u \in N(v)\})$$

Agregación Cálculo de mensaje

$$\text{AGG} = \text{LSTM}([\mathbf{h}_u^{(l-1)}, \forall u \in \pi(N(v))])$$

Agregación

Código*#SageConv*

GNN clásicas: GAT

Graph Attention Networks incorporan un factor de peso (importancia) sobre el mensaje de un nodo a otro (α_{vu})

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

Pesos de atenciones

No todos los nodos son igual de importantes:

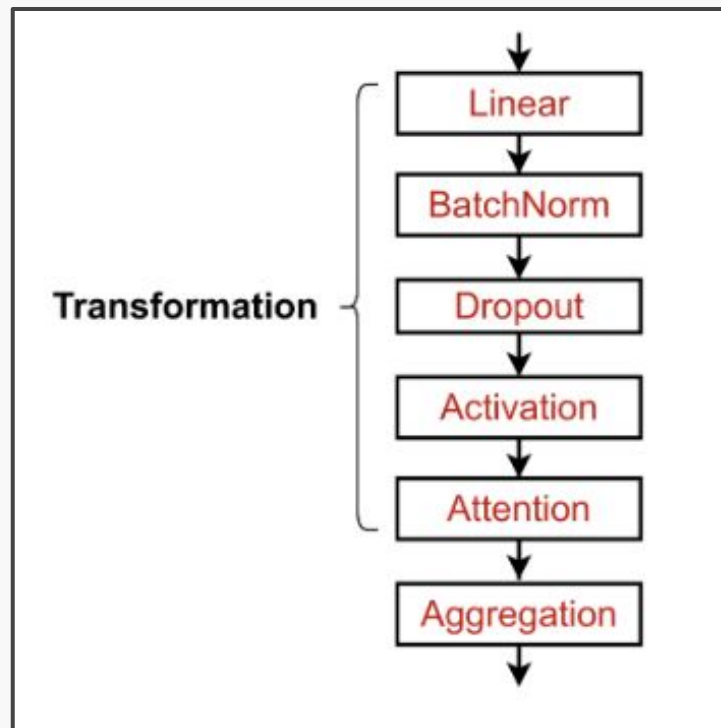
- La atención se enfoca en las partes importantes del *input*
- Al entrenar se aprende cuáles partes de los datos son más importantes

Más sobre implementación

En la práctica

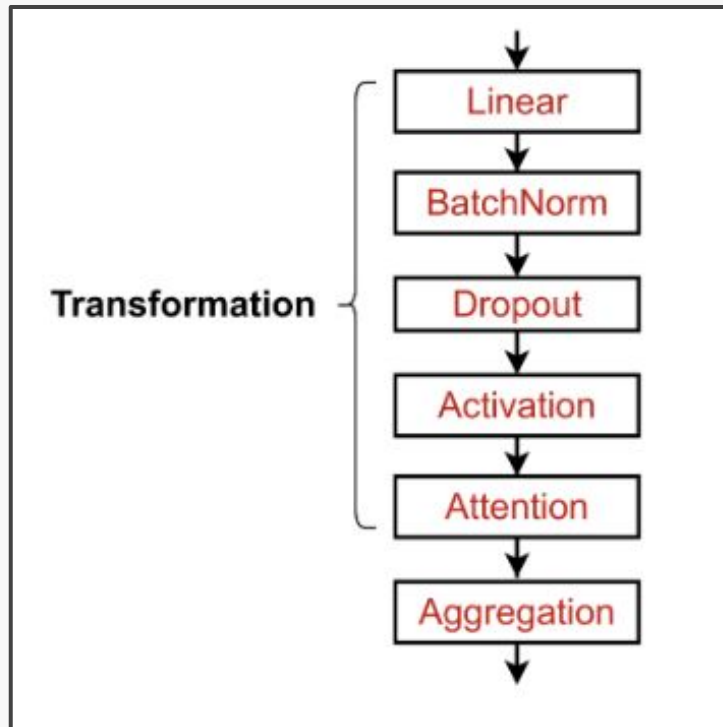
En la práctica, estas aproximaciones “clásicas” son un buen punto de inicio:

- Podemos mejorar la performance utilizando otros elementos de NNs en conjunto
- En concreto, podemos usar módulos modernos de DL en nuestras GNNs para mejorar nuestros modelos



En la práctica

- **Batch normalization:** Recentrar y escalar los embeddings (media cero, varianza uno)
- **Dropout:** Regularizar un modelo apagando neuronas al azar
- **Attention/gating**
- Y cualquier otro módulo



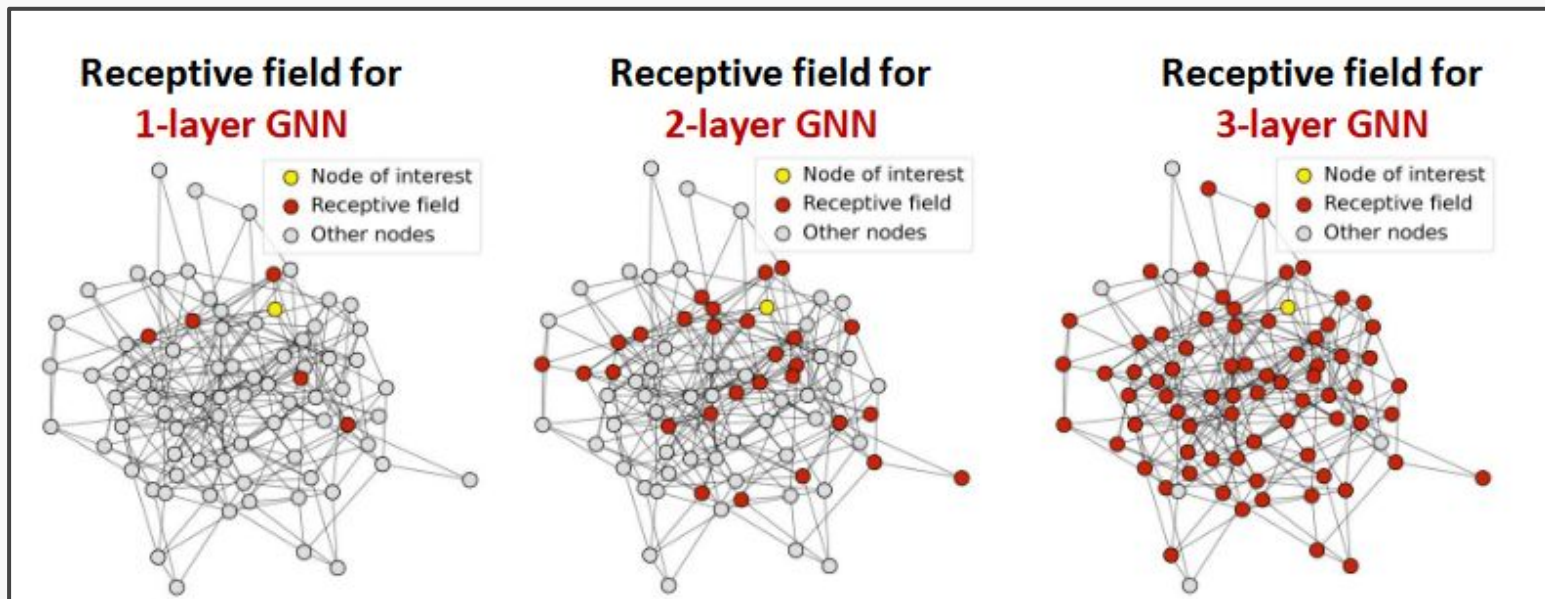
Apilando capas en una GNN

Podemos colocar capas especializadas para grafos una seguida de la otra, de la misma manera en que construimos modelos de DL en otros dominios.

¿Por qué NO hacer modelos tan profundos?

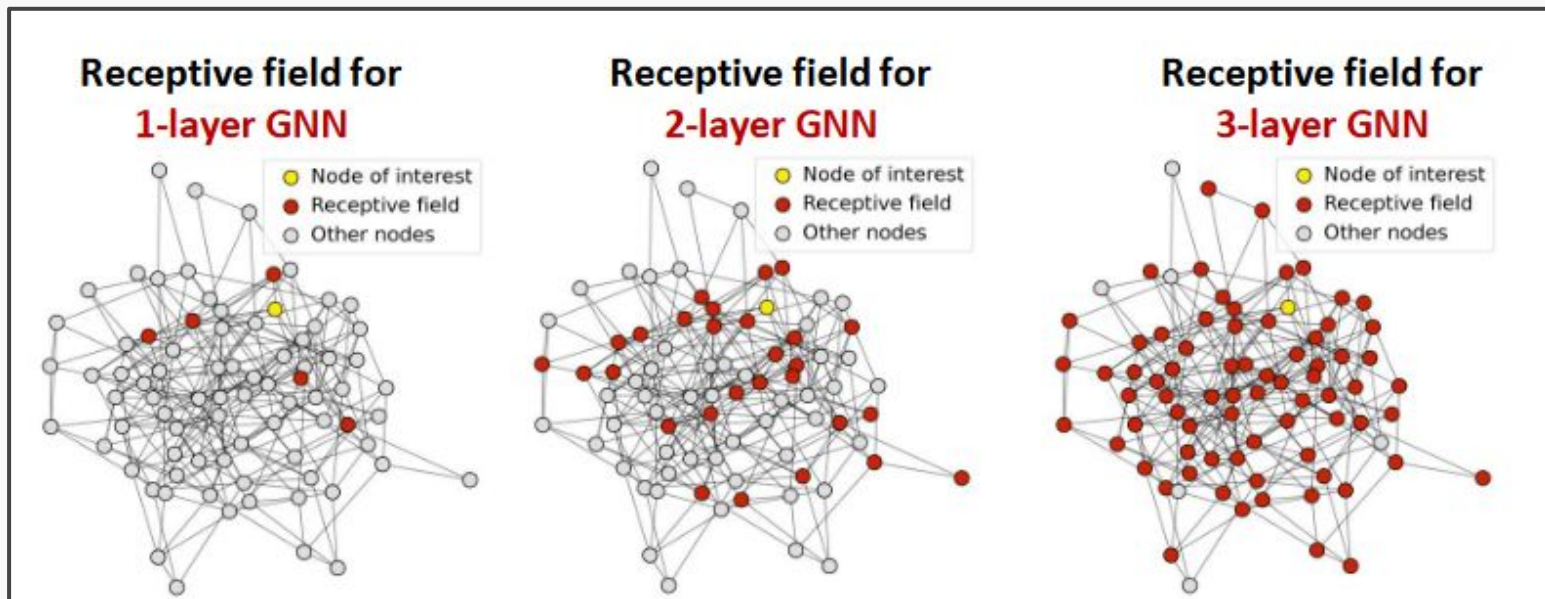
Over-smoothing problem

Al hacer un modelo muy profundo, con muchas capas de GNN, sufrimos este problema: todos los *embeddings* de nodos convergen al mismo valor



Over-smoothing problem

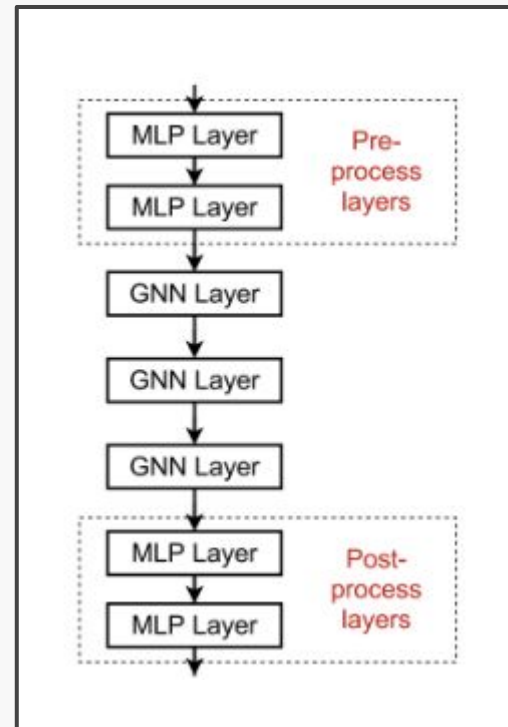
Esto ocurre porque, a medida que aumentamos la profundidad de un modelo, la cantidad de vecinos considerados en cada iteración aumenta y tiende a coincidir



Posibles medidas

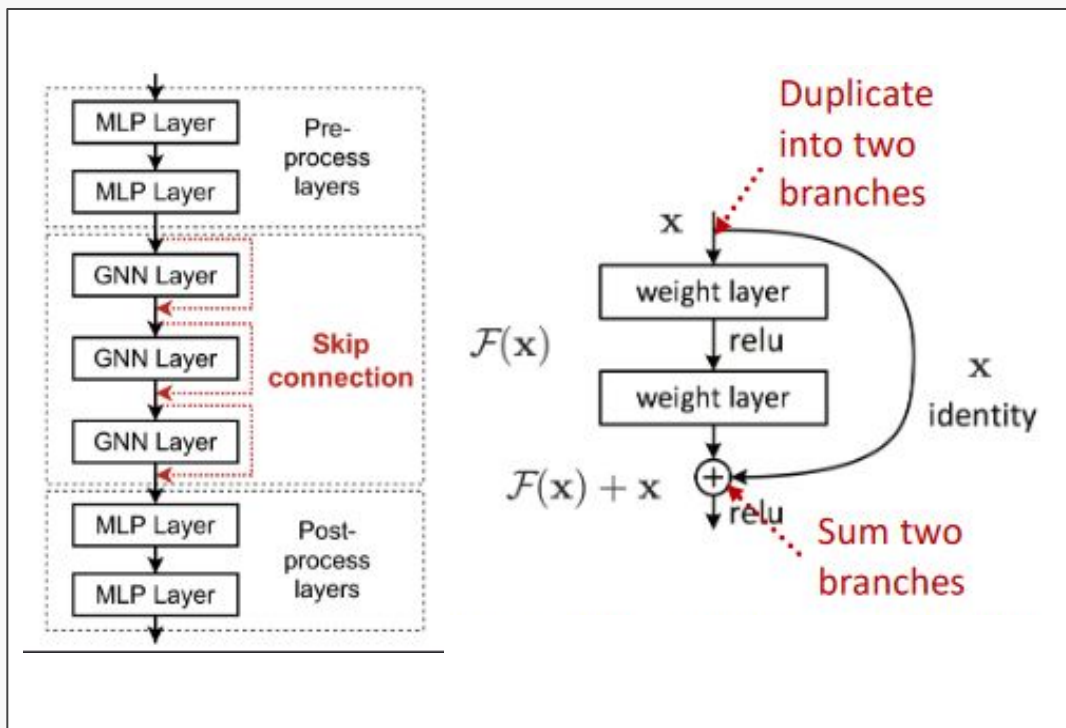
Solución 1: Aumentar la capacidad de cómputo dentro de cada capa, por ejemplo, convirtiendo cada operación de transformación o agregación en una pequeña red

Solución 2 (figura): Agregar capas que no pasen mensaje, para pre-procesamiento (antes de las capas GNN) como post-procesamiento (después de las capas GNN)



Posibles medidas

Solución 3 (figura): Agregar conexiones que se salten parte de las GNN. Esto permite que no se difuminen (tanto) las representaciones intermedias y que estén disponibles para cálculos en capas más cercanas a la salida



Clase 6.2

Un *framework* para GNNs

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl